



## **4. JDBC**

*Celso Paím*



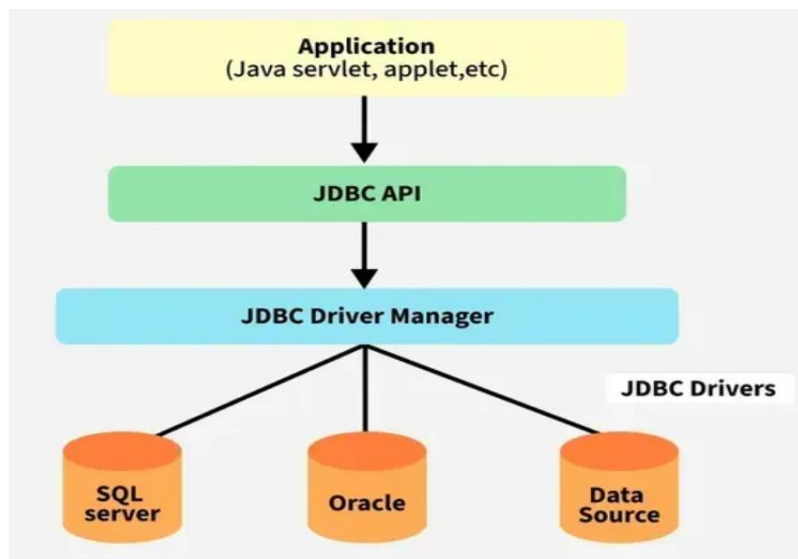
# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

## 4. JDBC

- O JDBC (Java Database Connectivity) é uma API Java de baixo nível que permite conectar aplicações Java a bases de dados relacionais (executadas na máquina local ou num servidor remotamente) ou outras fontes de dados.
- Ela actua como uma ponte, convertendo comandos nativos do Java em instruções SQL universais e gerindo a comunicação entre a aplicação e a base de dados.





# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

## 4.1 Conceitos de suporte

- **Base de dados:** um conjunto de dados (colecção de registos) que de uma forma estruturada ou não, organizam informação utilizada para uma finalidade comum. Podem ser tipificadas em:
  - OLTP
  - OLAP
  - No-SQL
- **SGBD (Sistema de Gestão de Base de Dados):** é um software que actua como intermediário entre a aplicação (ou utilizador) e os dados bruto.





## 4.1 Conceitos de suporte

- **MER (Modelo-Entidade Relação):** teoria matemática desenvolvida por Edgar Frank Codd.
  - Tabelas
  - Atributos
  - Tuplas
  - Relacionamentos
  - Cardinalidade
  - Chaves
  - Normalização
  - DER (Diagrama Entidade-Relação)



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

## 4.1 Conceitos de suporte

- **SQL (Structured Query Language)** é a linguagem padrão para acessar e manipular bases de dados. É transversal aos programadores típicos (desenvolvedores de aplicações) e aos arquitectos e administradores de bases de dados, independente do SGBD adoptado.
- Divide-se em 4 grandes grupos de comandos:
  - **DDL (Data Definition Language):** exemplo CREATE, DROP ou ALTER.
  - **DCL (Data Control Language) :** exemplo: GRANT e REVOKE.
  - **DML (Data Manipulation Language):** configuram o CRUD, com operações INSERT, DELETE, UPDATE e SELECT.
  - **DTL (Data Transaction Language) :** implemetam o ACID, com operações como BEGIN, ROLLBACK ou COMMIT.



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

## 4.2 Principais classes da API JDBC

- A API é composta pelos pacotes `java.sql` e `javax.sql`, e por meio das classes e interfaces fornecidas por esses dois pacotes, os programadores podem desenvolver softwares que acessem qualquer fonte de dados, desde bases de relacionais até planilhas.
- As principais classes do pacote `java.sql` agrupadas por propósito são:
  - Conexão: `DriverManager` e `Connection`, estabelecem conexões com a base de dados através de *drivers*.
  - Comandos: as classes `Statement`, `PreparedStatement` (para consultas parametrizadas) e `CallableStatement` (para chamadas de *procedures*).
  - Dados: a classe `ResultSet` armazena e manipula os dados retornados de consultas SQL.
- Já o pacote `javax.sql` contém classes mais avançadas com recurso a JNDI, gerir pool de conexões, transações distribuídas e `RowSets`.



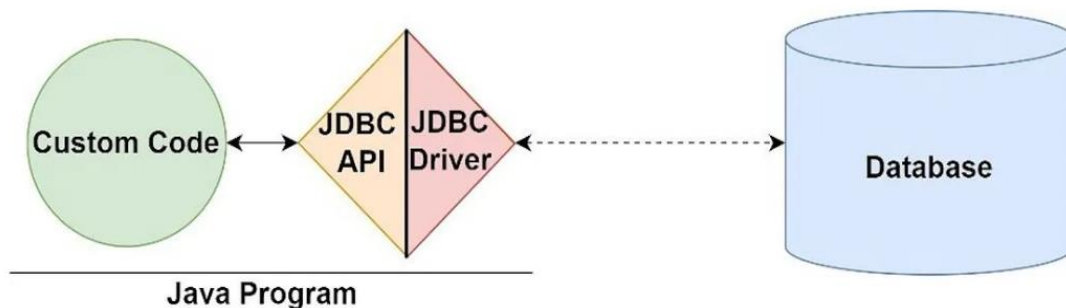
# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

#### 4.2.1 Drivers JDBC

- Um *driver* JDBC é uma classe que implementa a interface `java.sql.Driver` e são verdadeiros responsáveis pela conexão e interação com um SGBD específico.
- A classe `DriverManager` define um conjunto básico de operações para a manipulação do *driver* adequado para a conexão com uma BD. Além disso, ela também é responsável por realizar a conexão inicial.





# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

#### 4.2.1 Drivers JDBC

- A arquitetura do JDBC possui quatro tipos de *drivers* diferentes:
  - a) **Tipo 1 (JDBC-ODBC bridge)**: possibilita o acesso a *drivers* do tipo ODBC, um padrão já consolidado para o acesso a bases de dados.
  - b) **Tipo 2 (Native API driver)**: implementa o protocolo do proprietário do SGBD. Ele transforma as chamadas JDBC em chamadas do SGBD com o uso da API proprietária.
  - c) **Tipo 3 (Middleware driver)**: faz a conversão das chamadas JDBC em outras chamadas do SGBD, que são direcionadas para uma camada intermediária de *software*, ou *middleware*. Assim, a chamada será convertida para o protocolo do SGBD.
  - d) **Tipo 4 (Pure Java driver)**: são escritos puramente em Java e implementam o protocolo proprietário do banco de dados. No geral, têm desempenho superior, já que acessam directamente o SGBD,





## 4.2.2 Conectando um programa Java com uma BD

1. Garantir que possui o *driver* adequado para interagir com o SGBD;
2. Instalar e configurar correctamente o ficheiro `jar` com as classes do *driver* na CLASSPATH do programa que vai desenvolver;
3. No programa, definir a classe que implementa o *driver* JDBC, no nosso caso vamos usar `org.postgresql.Driver` para conectar com uma BD PostgreSQL;
4. No programa definir a *string* de conexão à base de dados. É importante mencionar que a maneira de definir esta *string* varia entre SGBD diferentes;
5. No programa fornecer nome de utilizador e senha para conectar à base de dados. A salientar que medidas de segurança adicionais devem ser adoptadas, tais como níveis de autorização na base de dados, encriptação das credenciais em ficheiros, esteganografia do código-fonte do programa, etc.

**A partir daí o programa já pode manipular dados na base de dados.**



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Maio/2026

// Objectivo: efectuar uma conexao a um servidor de BD PostgreSQL

```
import java.sql.*;
```

```
public class DemoConexaoPostgreSQL {  
    public static void main( String[] args ) {
```

**DemoConexaoPostgreSQL.java**

**Connection  
DriverManager**

```
        final String URL = "jdbc:postgresql://localhost:5432/livros";
```

```
        final String USER = "postgres";                //pratica pouco segura
```

```
        final String PASSWD = "ifsadmin";              //pratica pouco segura
```

```
    try {
```

```
        Class.forName( "org.postgresql.Driver" );
```

```
        Connection conexao = DriverManager.getConnection( URL, USER, PASSWD );
```

```
        System.out.println( "Conexao estabelecida com sucesso" );
```

```
        conexao.close();
```

```
    }
```

```
    catch ( ClassNotFoundException e ) {
```

```
        System.err.println( "Aconteceu um erro ao localizar o driver: " + e.getMessage() );
```

```
    }
```

```
    catch ( SQLException e ) {
```

```
        System.err.println( "Aconteceu um erro ao conectar: " + e.getMessage() );
```

```
    }
```

```
 }
```

```
}
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Maio/2026

// Objectivo: efectuar uma conexao a um servidor de BD MySQL

```
import java.sql.*;
```

```
public class DemoConexaoMySQL {  
    public static void main( String[] args ) {  
        final String URL = "jdbc:mysql://localhost:3306/livrosbd";  
        final String USER = "root"; //pratica pouco segura  
        final String PASSWD = "dbDoctor7"; //pratica pouco segura  
  
        try {  
            Class.forName( "com.mysql.jdbc.Driver" );  
            Connection conexao = DriverManager.getConnection( URL, USER, PASSWD );  
            System.out.println( "Conexao estabelecida com sucesso" );  
            conexao.close();  
        }  
        catch ( ClassNotFoundException e ) {  
            System.err.println( "Aconteceu um erro ao localizar o driver: " + e.getMessage() );  
        }  
        catch ( SQLException e ) {  
            System.err.println( "Aconteceu um erro ao conectar: " + e.getMessage() );  
        }  
    }  
}
```

**DemoConexaoMySQL.java**

**Connection  
DriverManager**



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Maio/2026

// Objectivo: inserir um registo e mostrar o conteudo uma tabela de uma BD PostgreSQL

```
import java.sql.*;
```

```
public class DemoManipularBDPostgreSQL {  
    public static void main( String[] args ) throws Exception {
```

```
        final String URL = "jdbc:postgresql://localhost:5432/livros";  
        final String USER = "postgres";           //pratica pouco segura  
        final String PASSWD = "ifsadmin";          //pratica pouco segura
```

```
        Connection conexao = DriverManager.getConnection( URL, USER, PASSWD );  
        lerTabelaAutores( conexao );
```

```
        System.out.println( "\nInserindo um novo registo na tabela..." );  
        inserirAutor( conexao, 5, "Agostinho", "Neto", new Date( 1924, 9, 17 ), 999 );
```

```
        lerTabelaAutores( conexao );
```

```
        conexao.close();
```

```
}
```

**DemoManipularBDPostgreSQL.java**

**Statement**

**PreparedStatement**



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

```
public static void lerTabelaAutores( Connection conexao ) {  
    try {  
        Statement stmt = conexao.createStatement();  
        String sqlQuery = "SELECT * FROM livro_schema.tb_autores ;";  
        ResultSet rs = stmt.executeQuery( sqlQuery );  
  
        System.out.println( "Tabela Autores\nID\tNome\tApelido\tData Nascimento\tTelefone" );  
        while ( rs.next() ) {  
            System.out.println( rs.getInt( "idAutor" ) + "\t"  
                + rs.getString( "nomeAutor" ) + "\t"  
                + rs.getString( "apelidoAutor" ) + "\t"  
                + rs.getDate( "dataNasc" ) + "\t"  
                + rs.getInt( "telefone" ) );  
        }  
        rs.close();  
        stmt.close();  
    }  
    catch ( SQLException e ) {  
        System.err.println( "Aconteceu um erro ao ler a tabela Autores: " + e.getMessage() );  
    }  
}
```

**DemoManipularBDPostgreSQL.java**

**Statement**

**PreparedStatement**



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

```
public static void inserirAutor( Connection conexao, int id, String nome, String apel, Date data, int tel ) {  
    try {  
        String sqlQuery = "INSERT INTO livro_schema.tb_autores VALUES (?,?,?,?,?)";  
        PreparedStatement ps = conexao.prepareStatement( sqlQuery );  
  
        ps.setInt( 1, id );  
        ps.setString( 2, nome );  
        ps.setString( 3, apel );  
        ps.setDate( 4, data );  
        ps.setInt( 5, tel );  
  
        int res = ps.executeUpdate();  
        if ( res == 1 )  
            System.out.println( "Registo inserido com sucesso!" );  
  
        ps.close();  
    }  
    catch ( SQLException e ) {  
        System.err.println( "Aconteceu um erro ao inserir o registo na tabela: " + e.getMessage() );  
    }  
}
```

**DemoManipularBDPostgreSQL.java**

**Statement**

**PreparedStatement**



## 4.3 Stored Procedures com JDBC

- Um *stored procedure* (ou procedimento armazenado) é um bloco de código contendo um ou mais comandos SQL, que é guardado diretamente no servidor de base de dados. As principais vantagens:
  - *Desempenho*: reduz o tráfego de dados na rede, já que a aplicação envia apenas um comando curto para o SGBD, e o processamento pesado ocorre inteiramente no servidor;
  - *Manutenção*: facilita a actualização de regras de negócio, que podem ser alteradas na base de dados sem a necessidade de recompilar ou reimplantar o código da aplicação.
  - *Segurança*: pode restringir o acesso direto dos usuários às tabelas, obrigando-os a interagir apenas através dos procedimentos, que validam e processam os dados de forma segura
- Para executar um *stored procedure* em Java via JDBC, utiliza-se a classe `CallableStatement`.



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Maio/2026

// Objectivo: invocar um stored procedure armazenado numa BD PostgreSQL

```
import java.sql.*;
```

```
public class DemoStoreProcedPostgreSQL {  
    public static void main( String[] args ) {
```

```
        final String URL = "jdbc:postgresql://localhost:5432/livros";
```

```
        final String USER = "postgres";           //pratica pouco segura
```

```
        final String PASSWD = "ifsadmin";         //pratica pouco segura
```

```
        try (
```

```
            Connection conn = DriverManager.getConnection( URL, USER, PASSWD ) ) {
```

```
            // preparar a chamada do stored procedure
```

```
            String sqlQuery = "{call consultarAutor(?, ?)}";
```

```
            CallableStatement stmt = conn.prepareCall( sqlQuery );
```

```
            // definir o parametro de entrada (IN)
```

```
            stmt.setInt( 1, 1 );
```

```
            // registar o parâmetro de saída (OUT)
```

```
            stmt.registerOutParameter( 2, java.sql.Types.VARCHAR );
```

**DemoStoreProcedPostgreSQL.java**

**Socket**





# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// executar o stored procedure

```
stmt.execute();
```

// reecuperar o valor de saída

```
String nome = stmt.getString( 2 );
```

```
System.out.println( "Nome do Autor: " + nome );
```

```
stmt.close();
```

```
}
```

```
catch ( SQLException e ) {
```

```
    System.err.println( "Aconteceu um erro ao invocar o stored procedure: " + e.getMessage() );
```

```
}
```

```
}
```

```
}
```



## **4.4 Bases de Dados No-SQL**

- Uma base de dados NoSQL (Not only SQL) é um tipo de base de dados que não se alicerça em tabelas relacionais como as bases de dados SQL tradicionais, utilizando modelos de dados mais flexíveis para armazenar dados estruturados, semi-estruturados e não-estruturados
- As bases de dados NoSQL não têm como objectivo substituir as bases de dados relacionais, mas apenas propor algumas soluções que em determinados cenários são mais adequadas. Desta forma é possível trabalhar tanto com bases de dados relacionais como com bases de dados NoSQL dentro de uma mesma aplicação.
- Java suporta praticamente todas as bases de dados No-SQL no mercado. Não com a API JDBC mas através de *drivers*.com APIs proprietárias dos fabricantes de SGBDs.



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Maio/2026

// Objectivo: conectar e manipular uma base de dados MongoDB simples via Java

```
import com.mongodb.*;  
import com.mongodb.client.*;  
import com.mongodb.client.MongoClient;  
import java.net.UnknownHostException;  
import org.bson.Document;
```

**DemoConexaoMongoDB.java**

```
public class DemoConexaoMongoDB {
```

```
    public static void main( String args[] ) throws UnknownHostException, MongoException {
```

```
        // abrir uma conexao com o servidor do MongoDB
```

```
        MongoClient mongoClient = MongoClient.create( "mongodb://localhost:27017" );
```

```
        // seleccionar a bd, que sera criada caso ainda nao exista
```

```
        MongoDB database = mongoClient.getDatabase( "contactos_bdNoSQL" );
```

```
        // seleccionar uma colecao para armazenar os dados e tambem a criar caso ainda nao exista
```

```
        MongoClient<Document> collection = database.getCollection( "contactos" );
```

```
        // criar um documento que ira representar um contacto
```

```
        Document doc1 = new Document();
```

```
        doc1.put( "id", 1 );
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

```
doc1.put( "nome", "Teresa Tati" );  
doc1.put( "email", "bt@gmail.com" );  
doc1.put( "telefone", 965300537 );
```

// criar um sub-documento, com a morada do contacto que na verdade e outro documento

```
Document doc1_morada = new Document( "rua", "Rua Duque de Chiazzi" )  
    .append( "numero", 220 )  
    .append( "cidade", "Menongue" );
```

// anexar o sub-documento ao documento

```
doc1.put( "endereco", doc1_morada );
```

// inserir o documento na colecao

```
collection.insertOne( doc1 );
```

// criar outro documento (representando outro contacto)

```
Document doc2 = new Document( "id", 2 )  
    .append( "nome", "Cidalia Ribeiro" )  
    .append( "email", "cr@gmail.com" )  
    .append( "telefone", 965300270 );
```

// inserir o outro documento na colecao, desta vez sem o campo morada

```
collection.insertOne( doc2 );
```

**DemoConexaoMongoDB.java**



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// consultar a BD procurando por algum contacto com base no seu telefone, ira retornar apenas o primeiro contacto que corresponda a condicao de pesquisa

```
Document docTemp = new Document();  
docTemp.put( "telefone", 965300270 );  
Document docEncontrado = collection.find( docTemp ).first();
```

// imprimir o objecto documento procurado (poderia invocar o println com docEncontrado.toJson() para imprimir no formato JSON)

```
System.out.println( docEncontrado );  
}  
}
```

**DemoConexaoMongoDB.java**